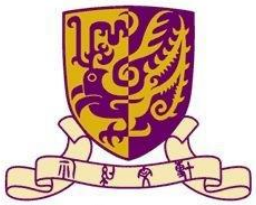


香港中文大學（深圳）
The Chinese University of Hong Kong

CSC 5003 Algorithm Art and Programming Practice
Lecture 4: Topsort and Shortest Path

Jingbang Chen
School of Data Science (SDS)
The Chinese University of Hong Kong, Shenzhen

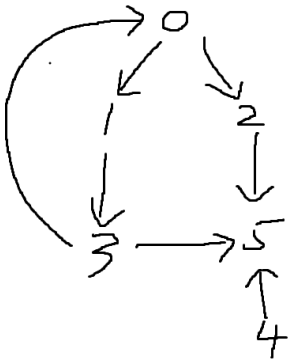
Find Eventual Safe States

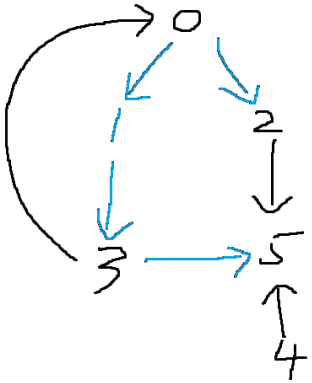
- Given a directed graph, a vertex is "safe" if and only if you cannot walk into a cycle from it.
- Output all "safe" vertices.
- This is called "flood-fill".

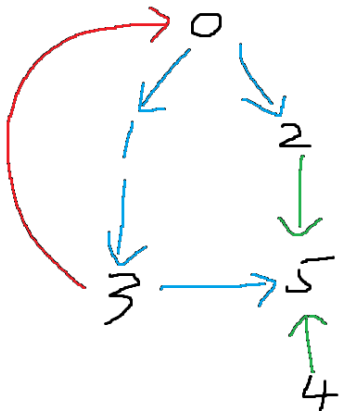
Solution

- Let safe[x] denote whether x is safe.
- We have safe[x] &= safe[y] (x → y)

- Consider a DFS-"flood-fill" on the graph, how many types of vertices we may visit (re-visit)
  - Unvisited
  - Visited but not an ancestor of x
  - Visited but an ancestor of x → a cycle is detected.
- **A DFS traversal determines a first-in-last-out order, breaking ties of siblings.**







Summary

- **A DFS traversal determines a first-in-last-out order, breaking ties of siblings.**

- **A BFS traversal decompose the graph in a first-in-first-out manner, following one of the following manners:**
  - Following the distance order from the source.
  - Indicating a topological order (in the directed graph case)

- → Topsort & Shortest Path

Topsort

- On a directed graph, x→y denotes that x must happen before y.
  - i.e. class perquisites
- BFS: Enqueue a vertex whenever there is not other in-edge (any vertex must enqueue before it)
  - Can output the smallest/largest lexicographic one
- DFS: The reversed order of any DFS-stack popping order.
  - Easy to implement (But don't forget to reverse it!)
- Only a DAG (Directed Acyclic Graph) has topological orders!

Course Schedule (15 mins)

- You are given an integer numCourses representing the total number of courses you have to take, labeled from 0 to numCourses - 1.
- You are also given an array prerequisites, where prerequisites[i] = [a, b] indicates that **to take course a, you must first take course b**.
- For example:
- [1, 0] means you must take course 0 before course 1.
- Return **true if you can finish all courses**, otherwise return **false**.

- **Hint: There are two ways of solving this problem!**

Shortest Path

- Single-source Multi-Destination:
  - Unweighted: BFS
  - Non-negative Weighted: Dijkstra
  - Any without Negative Cycles: Bellman-Ford
- Multi-source Multi-Destination: Floyd
- What are the complexity of these algorithms?

Rotting Oranges (20 mins)

- You are given an m x n grid where each cell can have one of three values:
- 0 represents an empty cell
- 1 represents a fresh orange
- 2 represents a rotten orange
- Every minute, **any fresh orange that is 4-directionally adjacent** (up, down, left, right) to a rotten orange becomes rotten.
- Return the **minimum number of minutes** that must elapse until **no cell has a fresh orange**.
  If this is impossible, return -1.

- Hint: Is this a single-source problem?

Bus Route (25 mins)

- You are given an array routes where routes[i] is a list of bus stops that **the i-th bus route** travels through.
- You can **get on or off a bus at any bus stop**.
- If you are on a bus, you can travel between any two stops on that same route.
- You are also given two integers source and target, representing the bus stop where you start and the bus stop you want to reach.
- Return the **minimum number of buses you must take** to travel from source to target.
  If it is not possible, return -1.

- Hint: How to build this graph to use the shortest path algorithm?

Huawei

Minimum Costs Using the Train Line (25 mins)

- You are given an integer n, representing n train stations labeled from 0 to n - 1.
- There are **two types of train routes**:
- 1 **Regular route**
- You can travel from station i to station i + 1
- Cost is regular[i]
- 2 **Express route**
- You can travel from station i to station j (j > i)
- Cost is expressCost + sum(express[i ... j-1])
- You must pay the **fixed express cost** every time you take an express route
- You start at station 0.
- Return an array answer of length n, where:
- answer[i] is the **minimum cost to reach station i from station 0**

Jump Game 1&2 (30 mins)

- You are given an integer array nums.
  You are initially positioned at the **first index** of the array.
- Each element nums[i] represents the **maximum jump length** you can take from index i.

- 1. Return **true if you can reach the last index**, otherwise return **false**.

- 2. Return the **minimum number of jumps** required to reach the last index.

- There are Jump Game 3&4……

Summary

- Topsort and Shortest Path are two common types of extension from traversal on graphs.
- Make sure you remember all algorithms
- The hardest part is about building the graph.
  - Identify which type of problem
  - Define the meaning of edges
  - Ensure any path/sequence indicate a legit solution
  - Apply a proper algorithm